



Artificial Intelligence BS(AI)
Shaheed Zulfiqar Ali Bhutto Institute of Science and Technology
(SZABIST)

Lab 4 & 5: Forward and Backward Propagation in Neural Network

Objectives:

- To explain the roles of forward propagation and backpropagation in training a neural network.
- To build a simple feedforward neural network using PyTorch's nn.Module class, load and preprocess a real-world image dataset (MNIST) using torchvision.
- To train a neural network using batches, epochs, a loss function, and an optimizer.
- To evaluate a trained model's accuracy on unseen test data and visualize training progress (loss curve) and model predictions on real images.

Software and Tools:

1.6 GHz or faster processor
4 GB of RAM
20 GB of available hard disk space
Windows 7 or later
Google Colab

Colab Notebook for Lab:

https://colab.research.google.com/drive/1sAfkVdUH20lj_7uHnwN9YaubbYqqliq?usp=sharing

Students should open the notebook, follow instructions, and perform the tasks to complete the lab exercises.

Background & Procedure:

A neural network is a function made up of layers of simple computational units (neurons), each of which applies a weighted sum of its inputs followed by a non-linear activation function. Stacking layers lets the network learn increasingly complex patterns in data. In this lab, the network takes a 28x28 pixel handwritten digit image and outputs a probability distribution over 10 possible digit classes (0 through 9).

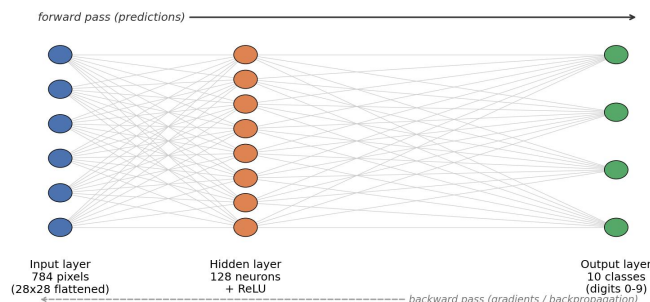


Figure 1: Architecture of the network used in this lab.



Artificial Intelligence BS(AI)
Shaheed Zulfikar Ali Bhutto Institute of Science and Technology
(SZABIST)

2.2 Forward Propagation

Forward propagation is the process of passing input data through the network, layer by layer, to produce an output. In this lab's network, the input image is first flattened from a 28x28 grid into a single vector of 784 values. This vector passes through a hidden layer of 128 neurons with a ReLU activation function, and finally through an output layer of 10 neurons, one score per digit class. The class with the highest score is the network's prediction.

2.3 Loss Function

The loss function measures how far the network's predictions are from the true labels. This lab uses Cross-Entropy Loss, the standard choice for multi-class classification problems. A high loss means the predicted probabilities are far from the correct answer; a low loss means the network is predicting confidently and correctly.

2.4 Backpropagation

Backpropagation is the algorithm that computes how much each weight in the network contributed to the loss, using the chain rule of calculus, working backward from the output layer to the input layer. In PyTorch, this entire process is triggered by a single line of code:

```
loss.backward()
```

PyTorch automatically tracks every operation performed on tensors during the forward pass (this is called autograd) and uses that record to compute the gradient of the loss with respect to every weight and bias in the network, without the student having to derive or code any calculus by hand.

2.5 Optimizer and Gradient Descent

Once gradients are computed, an optimizer uses them to update the network's weights so that the loss decreases. This lab uses the Adam optimizer, a widely-used variant of gradient descent that adapts the learning rate for each parameter automatically.

2.6 Epochs vs. Batches

An epoch is one complete pass through the entire training dataset. Because MNIST contains 60,000 training images, too many to process at once, the data is split into smaller batches (64 images at a time in this lab). The network's weights are updated after every batch, and one epoch consists of looping through all batches once. This lab trains for 3 epochs.

3. Dataset: MNIST

MNIST (Modified National Institute of Standards and Technology database) is a classic benchmark dataset in machine learning, consisting of 70,000 grayscale images of handwritten digits (0 to 9), each 28x28 pixels in size.



Artificial Intelligence BS(AI)
Shaheed Zulfikar Ali Bhutto Institute of Science and Technology
(SZABIST)

Training images	60,000
Test images	10,000
Image size	28 x 28 pixels, grayscale
Classes	10 (digits 0 through 9)
Pixel value range	0–255 (scaled to 0–1)

Import Libraries

Import PyTorch, torchvision (for the dataset), and matplotlib (for plotting).

```
import torch
import torch.nn as nn
import torchvision
import torchvision.transforms as transforms
import matplotlib.pyplot as plt
```

Load the MNIST Dataset

Download MNIST and wrap it in DataLoader objects, which handle batching and shuffling automatically.

```
transform = transforms.ToTensor()

train_dataset = torchvision.datasets.MNIST(root='./data', train=True, download=True,
transform=transform)
test_dataset = torchvision.datasets.MNIST(root='./data', train=False, download=True,
transform=transform)

train_loader = torch.utils.data.DataLoader(train_dataset, batch_size=64, shuffle=True)
test_loader = torch.utils.data.DataLoader(test_dataset, batch_size=1000, shuffle=False)
```

Expected output: "training images: 60000" and "test images: 10000".



Artificial Intelligence BS(AI)
Shaheed Zulfikar Ali Bhutto Institute of Science and Technology
(SZABIST)

Visualize Sample Images

Before training, display a few real images with their labels to confirm the data loaded correctly.

```
images, labels = next(iter(train_loader))

fig, axes = plt.subplots(1, 5, figsize=(8, 2))
for i, ax in enumerate(axes):
    ax.imshow(images[i][0], cmap="gray")
    ax.set_title(f'label: {labels[i].item()}')
    ax.axis("off")
plt.show()
```

Expected output: a row of 5 grayscale digit images, each captioned with its correct label.

Define the Model

Define a simple feedforward network with one hidden layer.

```
class Model(nn.Module):
    def __init__(self, in_features=784, hidden=128, out_features=10):
        super().__init__()
        self.fc1 = nn.Linear(in_features, hidden)
        self.fc2 = nn.Linear(hidden, out_features)

    def forward(self, x):
        x = x.view(x.size(0), -1)
        x = torch.relu(self.fc1(x))
        x = self.fc2(x)
        return x

torch.manual_seed(0)
model = Model()
print(model)
```

Expected output: a printed summary listing the two Linear layers (fc1 and fc2) with their dimensions.

Define Loss Function and Optimizer

```
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
```



Artificial Intelligence BS(AI)
Shaheed Zulfikar Ali Bhutto Institute of Science and Technology
(SZABIST)

Train the Model

Run the training loop for 3 epochs. For every batch, the network performs 4 steps:

1. Forward pass: compute predictions for the batch.
2. Compute loss: compare predictions to true labels.
3. Backward pass: `loss.backward()` computes gradients (backpropagation).
4. Update weights: `optimizer.step()` applies the gradients.

```
epochs = 3

for epoch in range(epochs):
    total_loss = 0
    for images, labels in train_loader:
        outputs = model(images)
        loss = criterion(outputs, labels)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

    total_loss += loss.item()

avg_loss = total_loss / len(train_loader)
print(f'epoch {epoch + 1} | avg loss {avg_loss:.4f}')
```

Expected output: the average loss printed once per epoch, decreasing each time (e.g. from around 0.4 down toward 0.1).

Evaluate on Test Data

```
correct = 0
total = 0

with torch.no_grad():
    for images, labels in test_loader:
        outputs = model(images)
        predictions = torch.argmax(outputs, dim=1)
        correct += (predictions == labels).sum().item()
        total += labels.size(0)

print(f'test accuracy: {correct / total:.4f}')
```

Expected output: a test accuracy typically above 0.95 (95%) after 3 epochs.



Artificial Intelligence BS(AI)
Shaheed Zulfikar Ali Bhutto Institute of Science and Technology
(SZABIST)

Visualize Predictions

```
images, labels = next(iter(test_loader))
with torch.no_grad():
    predictions = torch.argmax(model(images), dim=1)

fig, axes = plt.subplots(1, 5, figsize=(8, 2))
for i, ax in enumerate(axes):
    ax.imshow(images[i][0], cmap="gray")
    color = "green" if predictions[i] == labels[i] else "red"
    ax.set_title(f'pred: {predictions[i].item()}', color=color)
    ax.axis("off")
plt.show()
```

Lab Tasks:

1. Change the number of hidden neurons from 128 to 32, and again to 256. How does test accuracy change? How does training time change?
2. Increase the number of epochs from 3 to 10. Does the loss keep decreasing, or does it plateau?
3. Replace the ReLU activation with torch.sigmoid or torch.tanh. Does training speed or final accuracy change?
4. Change the optimizer from Adam to torch.optim.SGD with a learning rate of 0.01. Compare the loss curve to the Adam version.
5. Add a second hidden layer (e.g. 784 -> 128 -> 64 -> 10). Does accuracy improve?

Submission:

- Submit the Colab notebook with all steps executed.
- Ensure the notebook is well-commented and demonstrates your understanding of each operation.